Compiladores e Intérpretes

Tema 3.2.5 – Analizador Sintáctico – Descendente – Estrategia Recursiva Simple

Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

2026

Estrategia Recursiva

- Método para **gramáticas LL(1)**
- La intuición **simular la derivación** mediante una cadena de **llamadas recursivas**
  - Cada **llamada** simula un **paso en la derivación**

S⇒E**$**⇒TE'**$**⇒**(**E**)**E'**$**⇒**(**TE'**)**E'**$**⇒**(id**E'**)**E'**$**⇒**(id)**E'**$**⇒**(id)+**TE'**$**⇒**(id)+num**E'**$**⇒**(id)+num$**

Cada "flechita" implica que hicimos una llamada

S → E**$**
E → T E'
E' → **+** T E' | ε
T → **(**E**)** | **id** | **num**

Estrategia Recursiva

- Método para **gramáticas LL(1)**
- La intuición **simular la derivación** mediante **llamadas recursivas**
  - Cada **llamada** simula un **paso en la derivación**

S E$ TE'$ (E)E'$ (TE')E'$ (idE')E'$ (id)E'$ (id)+TE'$ (id)+numE'$ (id)+num$

Cada "flechita" se implica que hicimos una llamada

El simbolo $ lo usamos para denotar el fin de archivo o programa

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id | num

Estrategia Recursiva

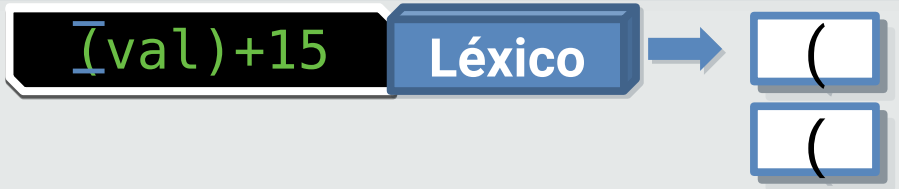
- Método para **gramáticas LL(1)**
- La intuición **simular la derivación** mediante una cadena de **llamadas recursivas**
  - Cada **llamada** simula un **paso en la derivación**

S E$ TE'$ (E)E'$ (TE')E'$ (idE')E'$ (id)E'$ (id)+TE'$ (id)+numE'$ (id)+num$

¿Qué representan los **métodos** llamados?

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num

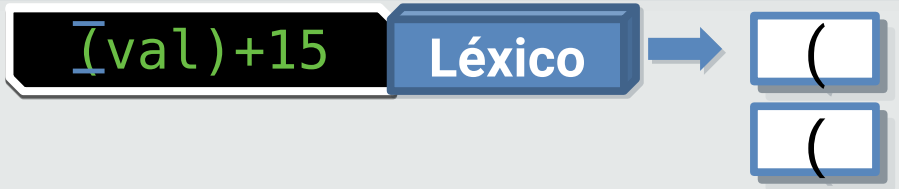




Veamos esto en base a como funciona un analizador sintáctico descendente como presentamos en el tema 3.2.1 y supongamos que estamos en esta situación

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num



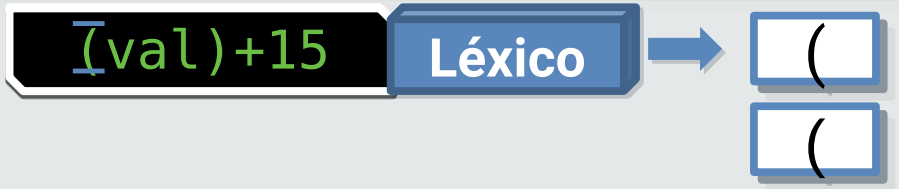




¿Qué representan los **métodos** llamados?

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num





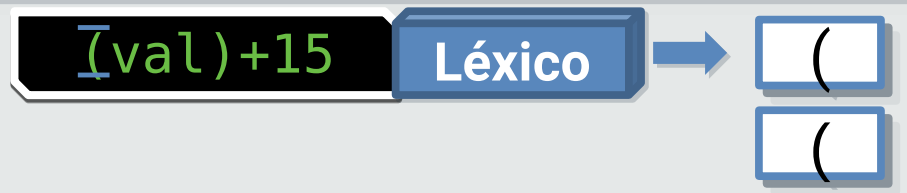


¿Qué representan los **métodos** llamados?

Los **no terminales** que vamos **expandiendo** en la derivación!

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num



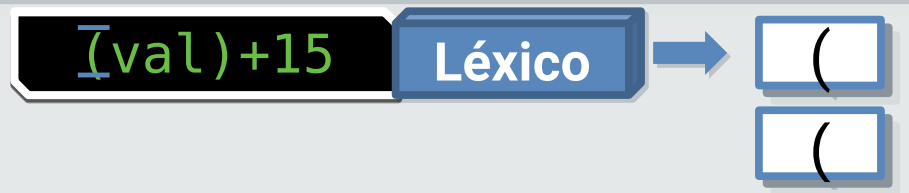




En este caso, si los NT son métodos, tendríamos que llamar al método correspondiente a T

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num



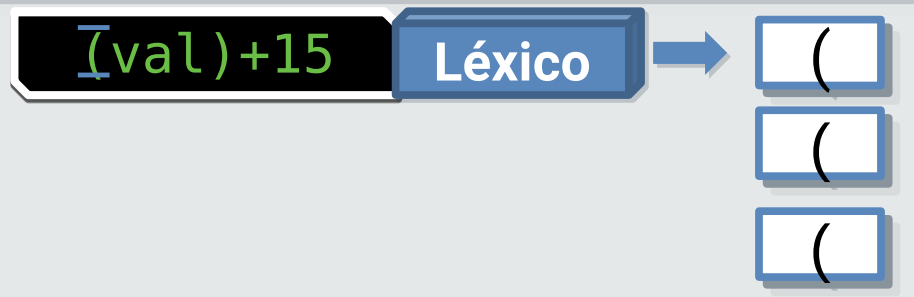


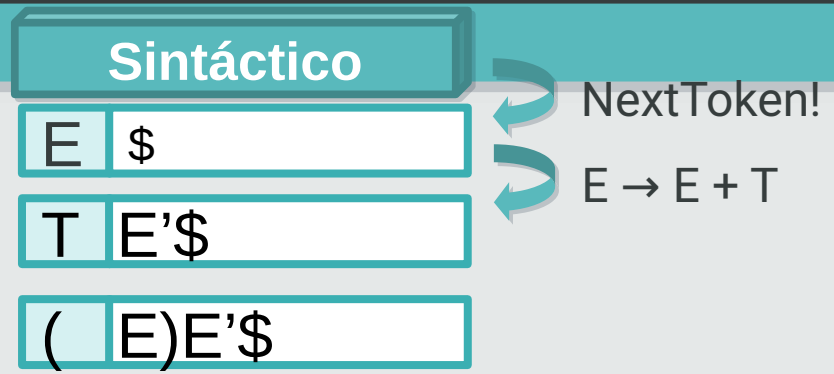


Mirando el token del léxico deberíamos poder elegir dentro del método de T por donde ir… En este caso la producción que va a (E)

Estrategia Recursiva

S → E$
E → T E’
E’ → + T E’
E’ → ε
T → (E)
T → id
T → num







Estando en el método de T deberíamos ver si matchea el token del léxico con el terminal del producción

Intuiciones para la Implementación del Analizador Sintáctico Descendente Recursivo

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Simulamos el comportamiento de la producciones para ese NT codificando sus partes derechas!

Vamos implementado en orden y para:
- Un **NT llamamos** al método de ese NT
- Un **Terminal** vemos si **machea** con el token actual

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Simulamos el comportamiento de la producciones para ese NT codificando sus partes derechas!

Vamos implementado en orden y para:
- Un **NT llamamos** al método de ese NT
- Un **Terminal** vemos si **machea** con el token actual

S → E$
E → T E'
E' → + T E'
E' → ε

```
void S(){
    ...
}
```

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Simulamos el comportamiento de la producciones para ese NT codificando sus partes derechas!

Vamos implementado en orden y para:
- Un **NT llamamos** al método de ese NT
- Un **Terminal** vemos si **machea** con el token actual

S → E$
E → T E'
E' → + T E'
E' → ε

```
void S(){
    E()
    ...
}
```

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Simulamos el comportamiento de la producciones para ese NT codificando sus partes derechas!

Vamos implementado en orden y para:
- Un **NT llamamos** al método de ese NT
- Un **Terminal** vemos si **machea** con el token actual

S → E$
E → T E'
E' → + T E'
E' → ε

```
void S(){
    E();
    match("$");
}
```

Intuiciones de Implementación – Llamadas

- En esta estrategia descendente vamos a desarrollar **un método** por **cada no terminal** de la gramática

¿Qué hacemos en el **cuerpo** del método para un NT?

Simulamos el comportamiento de la producciones pa
NT codificando sus partes derechas!

Veamos que es este “match”

Vamos implementado en orden y para:
- Un **NT llamamos** al método de ese NT
- Un **Terminal** vemos si **machea** con el token actual

S → E$
E → T E’
E’ → + T E’
E’ → ε

```
void S()
    E();
    match("$");
}
```

Intuiciones de Implementación – Match

- El **match** es un método que nos da la funcionalidad para ver si el **token actual** que nos envió el léxico **coincide** con el tipo de **token que esperamos**
  - Además, en el caso de que coincidan, como vimos en 3.2.1 tenemos que **pedir un nuevo token al léxico**

```
void match(String nombreToken){
    if(nombreToken.equals(tokenActual.nombre)){
        tokenActual = analizadorLexico.proximoToken();
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Match

- El **match** es un método que nos da la funcionalidad para ver si el **token actual** que nos envió el léxico **coincide** con el tipo de **token que esperamos**
  - Además, en el caso de que coincidan, como vimos en 3.2.1 tenemos que **pedir un nuevo token al léxico**

¿Que pasa si no coinciden?

```
void match(String nombreToken){
    if(nombreToken.equals(tokenActual.nombre)){
        tokenActual = analizadorLexico.proximoToken();
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Match

- El **match** es un método que nos da la funcionalidad para ver si el **token actual** que nos envió el léxico **coincide** con el tipo de **token que esperamos**
  - Además, en el caso de que coincidan, como vimos en 3.2.1 tenemos que **pedir un nuevo token al léxico**

```
void match(String nombreToken){
    if(nombreToken.equals(tokenActual.nombre)){
        tokenActual = analizadorLexico.proximoToken();
    }else{
        throw new ExcepcionSintactica(tokenActual, nombreToken);
    }
```

Se produce un **error** sintactico!

Básicamente, lo que paso es que en el programa hay algo distinto a lo que esperábamos en el punto exacto que tratamos de hacer el match

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación de su método?

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación de su método?

En el cuerpo vamos a implementar todas las partes derechas de las producciones del NT…

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación de su método?

En el cuerpo vamos a implementar todas las partes derechas de las producciones del NT…

E' → + T E'
E' → ε
T → (E)
T → id
T → num

```
void T(){
    ...
        match("(");
        E();
        match(")");
    ...
        match("id");
    ...
        match("num")
    ...
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación de su método?

En el cuerpo vamos a implementar todas las partes derechas de las producciones del NT…

Como vimos en el funcionamiento intuitivo del sintáctico la idea es elegir una de las producciones a aplicar en base a **Primeros**…
¿Como haríamos esto en el código?

$E' \rightarrow + T\ E'$
$E' \rightarrow \varepsilon$
$T \rightarrow (E)$
$T \rightarrow id$
$T \rightarrow num$

```
void T(){
    ...
        match("(");
        E();
        match(")");
    ...
        match("id");
    ...
        match("num")
    ...
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación de su método?

En el cuerpo vamos a implementar todas las partes derechas de las producciones del NT…

E' → + T E'
E' → ε
T → (E)
T → id
T → num

```
void T(){
    if(tokenActual.nombre.equals("(")){
        match("(");
        E();
        match(")");
    }else if(tokenActual.nombre.equals("id")){
        match("id");
    }else if(tokenActual.nombre.equals("num")){
        match("num")
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT t
¿Como lo reflejamos en la im

En el cuerpo vamos a implementar toda s derechas de las producciones del NT...

E' → + T E'

E' → ε

T → (E)

T → id

T → num

La **idea intuitiva** es elegir según el **token actual** mirando el **primer token** que nos podría **aparecer** si aplicamos esa **producción**

```
void T(){
    if(tokenActual.nombre.equals("(")){
        match("(");
        E();
        match(")");
    }else if(tokenActual.nombre.equals("id")){
        match("id");
    }else if(tokenActual.nombre.equals("num")){
        match("num")
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

Es decir, miramos si el **token actual** esta en **primeros del cuerpo** de la **regla** que vamos a **elegir**

La **idea intuitiva** es elegir según el **token actual** mirando el **primer token** que nos podría **aparecer** si aplicamos esa **producción**

E' → + T E'

E' → ε

T → (E)

T → id

T → num

```
void T(){
    if(tokenActual.nombre.equals("(")){
        match("(");
        E();
        match(")");
    }else if(tokenActual.nombre.equals("id")){
        match("id");
    }else if(tokenActual.nombre.equals("num")){
        match("num")
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
implementación de su método?

¿Qué paso y qué deberíamos hacer si el **token actual no coincide** con **ninguna** de las opciones para elegir la **producción** adecuada?

mentar todas las partes derechas de
cciones del NT…

E' → ε

T → (E)
T → id
T → num

```
void T(){
    if(tokenActual.nombre.equals("(")){
        match("(");
        E();
        match(")");
    }else if(tokenActual.nombre.equals("id")){
        match("id");
    }else if(tokenActual.nombre.equals("num")){
        match("num")
    }else{
        ...
    }
}
```

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas de una producción?
¿Como lo reflejamos en la implementación?

En el cuerpo vamos a implementar todas las producciones del NT...

E' → + T E'

E' → ε

T → (E)

T → id

T → num

```
void T(){
    if(tokenActual.nombre.equals("(")){
        match("(");
        E();
        match(")");
    }else if(tokenActual.nombre.equals("id")){
        match("id");
    }else if(tokenActual.nombre.equals("num")){
        match("num")
    }else{
        throw new ExcepcionSintactica(tokenActual, "(, id o num");
    }
}
```

Básicamente es otra forma de que se produzca un **error sintactico**

Intuiciones de Implementación – Múltiples Producciones de un NT

¿Que hacemos cuando un NT tiene mas
¿Como lo reflejamos en la implement

En el c

Si el nro de línea y el lexema del token actual son N y X respectivamente, reportaríamos:

"Error sintáctico en línea N: se encontro X cuando se esperaba un (, id o num"

Básicamente es otra forma de que se produzca un **error sintactico**

E' → ε

T → (E)
T → id
T → num

```
e.equals("(")){
    match("(");
    E();
    match(")");
}else if(tokenActual.nombre.equals("id")){
    match("id");
}else if(tokenActual.nombre.equals("num")){
    match("num")
}else{
    throw new ExcepcionSintactica(tokenActual, "(, id o num");
}
}
```

Esquema General de Implementación - Simple

Esquema Simple de Implementación

- Este esquema de implementación se vale de:
  - Las **intuiciones generales** que *vimos antes* y
  - La noción de **primeros** para elegir cuando hay **múltiples producciones** para un NT

Esquema Simple de Implementación

- Este esquema de implementación se vale de:
  - Las **intuiciones generales** que *vimos antes* y
  - La noción de **primeros** para elegir cuando hay **múltiples producciones** para un NT

La idea que cuando tengo varias producciones para un NT, voy a **elegir** aquella que **contenga** al **token actual** en los **primeros de su parte derecha**

Esquema Simple de Implementación

- Dada una gramática y un **no terminal A** donde sus **producciones** tienen la forma:

$$A \rightarrow \beta_1$$
$$A \rightarrow \beta_2$$
$$\ldots$$
$$A \rightarrow \beta_n$$

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de Implementación

- Dada una gramática y un **no terminal A** donde sus **producciones** tienen la forma:

$A \rightarrow \beta_1$

$A \rightarrow \beta_2$

...

$A \rightarrow \beta_n$

SI **ningún Primeros** de las partes derechas **tiene a ε** el esquema es...

```
void A(){
    if(tokenActual esta en Primeros(β1))
        //traducción de β1
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    ...
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    else
        //error
}
```

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de Imple

Estas traducciones son como vimos antes: hacemos match para los terminales y llamamos a los no terminales

- Dada una gramática y un **no terminal A** de[illegible] **producciones** tienen la forma:

$A \rightarrow \beta_1$

$A \rightarrow \beta_2$

...

$A \rightarrow \beta_n$

SI **ningún Primeros** de las partes derechas **tiene a ε** el esquema es...

```
void A(){
    if(tokenActual esta en Primeros(β1))
        //traducción de β1
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    ...
    else if(tokenActual esta en Primeros(βn))
        //traducción de βn
    else
        //error
}
```

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de Implementación

- Dada una gramática y un **no terminal A** cuyas **producciones** tienen la forma:

$A \rightarrow \beta_1$
$A \rightarrow \beta_2$
...
$A \rightarrow \beta_n$

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

SI **ningún Primeros** de las partes derechas **tiene a ε** el esquema es...

Ninguno de estos tiene que tener a **ε**!!!!

```
void A(){
    if(tokenActual esta en Primeros(β1))
        //traducción de β1
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    ...
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    else
        //error
}
```

Esquema Simple de Implementación

¿Como sería el esquema si alguno tiene ε?

Ninguno de estos tiene que tener a ε!!!!

- Dada una gram… **…minal …** **producciones** tienen la forma:

$A \rightarrow \beta_1$
$A \rightarrow \beta_2$
…
$A \rightarrow \beta_n$

SI **ningún Primeros** de las partes derechas **tiene a ε** el esquema es...

```
void A(){
    if(tokenActual esta en Primeros(β1))
        //traducción de β1
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    …
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    else
        //error
}
```

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de Implementación

- Dada una gramática y un **no terminal A** donde sus **producciones** tienen la forma:

$A \rightarrow \beta_1$
$A \rightarrow \beta_2$
...
$A \rightarrow \beta_n$

SI **algun** **Primeros** de las partes derechas **tiene a ε** el esquema es...

```
void A(){
    if(tokenActual esta en Primeros(β1))
        //traducción de β1
    else if(tokenActual esta en Primeros(β2))
        //traducción de β2
    ...
    else if(tokenActual esta en Primeros(βn))
        //traducción de βn
    else
        //No hago nada :)
}
```

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de I

- Dada una gramática y un **no terr**
  producciones tienen la forma:

$A \rightarrow \beta_1$
$A \rightarrow \beta_2$
...
$A \rightarrow \beta_n$

SI **algun** **Primeros** de las partes derechas **tiene a ε** el esquema es...

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

En este caso, en el esquema simple, como algun β puede llevarnos a **vacío**, el **tokenActual no va coincidir** con ninguno de los **primeros** y no va ser un error, simplemente **no hacemos nada...** Esto captura la situacion de que desde A se llego a **ε**

```
        //                    de β1
else if(                    al esta en Primeros(β2))
        //tr                  de β2
...
else if(tok                 tual esta en Primeros(β2))
        //tradu               ón de β2
else
        //No hago nada :)
}
```

Esquema Simple de I

- Dada una gra… **producciones** …

En el esquema avanzado esto se trata de otra forma… pero eso lo vamos a ver en tema siguiente

En este caso, en el esquema simple, como algun β puede llevarnos a **vacío**, el **tokenActual no va coincidir** con ninguno de los **primeros** y no va ser un error, simplemente **no hacemos nada…** Esto captura la situacion de que desde A se llego a **ε**

$A \rightarrow \beta_1$

$A \rightarrow \beta_2$

…

$A \rightarrow \beta_n$

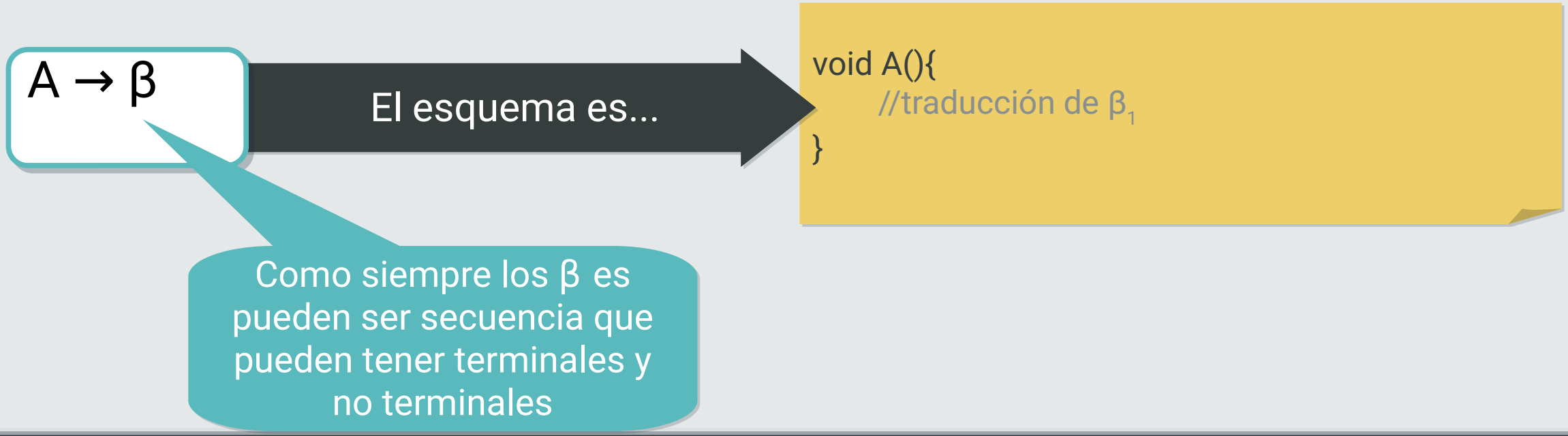
SI **algun** **Primeros** de las partes derechas **tiene a ε** el esquema es...

```
…de β1
else if(…al esta en Primeros(β2))
    //tr… de β2
…
else if(tok…tual esta en Primeros(β2))
    //tradu…ón de β2
else
    //No hago nada :)
}
```

Como siempre los β es pueden ser secuencia que pueden tener terminales y no terminales

Esquema Simple de Implementación

- Por último, si **A** tiene **una única producción** de la forma



Ejemplo de un Analizador Sintáctico Descendente usando el Esquema Simple

Esquema Simple de Implementación - Ejemplo

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id | num

```
class AnalizadorSintactico{
    AnalizadorLexico analizadorLexico;
    Token tokenActual;

    void match(String nombreToken){
        if(nombreToken.equals(tokenActual.nombre)){
            tokenActual = analizadorLexico.proximoToken();
        }else{
            throw new ExcepcionSintactica(tokenActual, nombreToken);
        }
    }

    ...

}
```

El analizador sintáctico va conocer al Léxico y va llevar registro del token actual, ademas de tener el método match que *vimos antes*

Esquema Simple de Implementación - Ejemplo

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id | num

```
class AnalizadorSintactico{
    AnalizadorLexico analizadorLexico;
    Token tokenActual;

    void match(String nombreToken){...}

    void S(){
        E();
        match("$");
    }

    void E(){
        T();
        Eprima();
    }

    ...
}
```

Los NT S y E tienen una sola producción asi que se traducen según el ultimo esquema

Esquema Simple de Implementación - Ejemplo

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id | num

```
class AnalizadorSintactico{
    AnalizadorLexico analizadorLexico;
    Token tokenActual;

    void match(String nombreToken){...}

    void S(){...}

    void E(){...}

    void Eprima(){
        //Primeros(+TE')={+}
        if(Arrays.asList("+").contains(tokenActual.nombre)){
            match("+");
            T();
            EPrima();
        }else{
            //No hago nada por que E'-> e
        }
    }

    ...
}
```

E' tiene una producción que nos lleva a ε por eso tenemos que usar el segundo esquema

Esquema Simple

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id | num

T no tiene ninguna producción que lleve ε por lo tanto usamos el primer esquema

```
class AnalizadorSintactico{
    AnalizadorLexico analizadorLexico;
    Token tokenActual;

    void match(String nombreToken){...}
    void S(){...}
    void E(){...}
    void Eprima(){...}

    void T(){
        //Primeros((E))={(}
        if(Arrays.asList("(").contains(tokenActual.nombre)){
            match("(");
            E();
            match(")");

        //Primeros(id)={id}
        }else if(Arrays.asList("id").contains(tokenActual.nombre)){
            match("id");

        //Primeros(num)={num}
        }else if(Arrays.asList("num").contains(tokenActual.nombre)){
            match("num");

        }else{
            throw new ExcepcionSintactica(tokenActual, "(, id o num");
        }
    }
}
```

Esquema Simple de Implementación - Ejemplo

S → E$
E → T E'
E' → + T E' | ε
T → (E) | id |

Por último, en el constructor recibimos la dependencia del Léxico, cargamos el primer token que nos da y llamamos al símbolo inicial de la gramática

```
class AnalizadorSintactico{
    AnalizadorLexico analizadorLexico;
    Token tokenActual;

    void match(String nombreToken){...}
    void S(){...}
    void E(){...}
    void Eprima(){...}
    void T(){...}

    AnalizadorSintactico(AnalizadorLexico ALex){
        analizadorLexico = ALex;
        tokenActual = analizadorLexico.proximoToken();
        S();
    }
}
```

Comentarios Finales

- Para poder implementar un analizador sintáctico usando esta estrategia es **necesario tener una gramática LL(1)**
  - Si la gramática **no es LL(1)** se le deben aplicar las tecnicas de **Eliminación de la recursion a izquierda** y **factorización** que vimos en los temas anteriores

Esta estrategia es la que vamos a usar en el **proyecto** para implementar el analizador sintactico

Hay una **versión avanzada** de esta estrategia (que también se puede usar para el proyecto) – Ver Material